

Pengantar Perkuliahan Algoritma & Struktur Data STI

IF2111 – Algoritma dan Struktur Data STI
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

Pendahuluan

IF2111/Algoritma & Struktur Data STI (3 SKS)

→ kuliah wajib bagi mahasiswa Prodi S1 STI.

Prasyarat IF2111:

- KU1102/Pengenalan Komputasi
(kurikulum lama: KU1072/Pengenalan Teknologi Informasi B)
- IF1210/Dasar Pemrograman

Tujuan Perkuliahan

Tujuan Instruksional Umum

Memberikan kemampuan untuk melakukan **pemrograman dalam skala menengah** dengan memanfaatkan **struktur data internal yang kompleks** dan **mengimplementasikan** dalam bahasa pemrograman yang dipilih.

Outcomes

Outcome yang diharapkan:

Mahasiswa mampu untuk membuat primitif ADT dan menggunakannya untuk program yang berstruktur data kompleks dalam paradigma prosedural dan mengimplementasi dalam bahasa pemrograman yang dipilih.

Outcomes (ABET)

Student Outcome ABET yang terkait:

- 1) Analyze a complex computing problem and to apply principles of computing and other relevant disciplines to identify solutions.
- 2) Design, implement, and evaluate a computing-based solution to meet a given set of computing requirements in the context of the program's discipline.
- 3) Function effectively as a member or leader of a team engaged in activities appropriate to the program's discipline.
- 4) Apply computer science theory and software development fundamentals to produce computing-based solutions.

Beban Perkuliahan

IF2111/Algoritma & Struktur Data STI → **3 SKS**:

- 3 jam **kuliah**
- 2 jam kegiatan terbimbing (**praktikum**)
- 4 jam kegiatan mandiri (**belajar mandiri/tugas/PR**)

Pelaksanaan Kuliah

Pertemuan tatap muka 2 kali seminggu:

Jadwal sesuai pengaturan prodi

Praktikum dilaksanakan 1 kali seminggu:

Jadwal sesuai pengaturan prodi

Kegiatan Praktikum

Bahasa Pemrograman: **Bahasa C**

Tujuan: memahami implementasi dan penggunaan ADT dalam program skala menengah.

Pra-praktikum: bila diperlukan akan diberikan tugas untuk diselesaikan secara mandiri **untuk digunakan saat praktikum** (sebagian besar dalam bentuk ADT).

Kegiatan **praktikum:** menyelesaikan tugas tertentu yang diberikan dalam 100 menit.

Mekanisme praktikum: akan dijelaskan oleh asisten (tbd).

Penilaian (1/2)

1. Ujian Tengah Semester (UTS)
2. Ujian Akhir Semester (UAS)
3. Kuis (jika diperlukan)
4. Tugas Besar
5. Hasil praktikum, termasuk Ujian Praktikum (jika ada)

Penilaian (2/2)

Tidak ada susulan dalam pengumpulan tugas, praktikum, dan kuis.

Ujian susulan (UTS, UAS) hanya diberikan jika ada alasan yang dapat diterima tim dosen mata kuliah dan diberitahukan sebelum jadwal ujian.

Informasikan sesegera mungkin ke dosen kelas (via email dan/atau langsung) jika ada kasus *emergency*, disertai surat keterangan pendukung yang sah → pertimbangan jika Anda di ambang tidak lulus untuk mendapatkan keringanan/remedial.

Tidak ada toleransi bagi pencontek pada pekerjaan apa pun → jika terbukti, langsung mendapat nilai E → baca kembali Peraturan Akademik ITB.

Materi Kuliah

Algoritma,
Struktur Data,
dan *Abstract Data Type* (ADT)

Struktur *array* dan struktur berkait

Sequence/list

~~Matriks~~

Stack

Queue

Set

Map

Tree,
binary tree,
binary search tree

Graph

Tata Laksana Kuliah

Semua hal terkait kuliah dilakukan di Google Classroom.

- Dosen dapat dihubungi via Google Chat atau email:
([riza](mailto:riza@staff.stei.itb.ac.id) | [farrell](mailto:farrell@staff.stei.itb.ac.id))@staff.stei.itb.ac.id
- Gunakan etika berkomunikasi yang baik.
- Google Chat Room (IF2111 Algoritma dan Struktur Data STI 2022) bisa digunakan juga

Semua hal terkait praktikum dilakukan sesuai arahan asisten.

Kuliah Tatap Muka (Virtual)

Dilakukan di Zoom.

Tata tertib:

- a) Masuk tepat waktu.
- b) Keterlambatan dan keluar/masuk saat kelas berlangsung harus memberitahu di *meeting chat, as soon as possible*.
- c) *Mute mic* kecuali saat berbicara.
- d) Daftar hadir diambil dari attendance list zoom
- e) Latihan soal dikerjakan secara individu kecuali ada instruksi khusus dari dosen.

Before & After Class

Sebelum setiap pertemuan Anda wajib menonton video kuliah dan membaca pdf slide untuk materi yang sesuai.

Setelah menonton video, kerjakan quiz kecil.

Link ke video & quiz ada di Google Classroom

Pustaka Wajib

Inggriani Liem, *“Diktat Struktur Data (Bagian I dan II)”*, 2003, Teknik Informatika.

Inggriani Liem, *“Catatan Singkat Bahasa C”*, Departemen Teknik Informatika ITB, 1998.

Inggriani Liem, *“Contoh Program Kecil dalam Bahasa C”*, Departemen Teknik Informatika ITB, 1998.

Inggriani Liem, *“Diktat Dasar Pemrograman, Bagian Pemrograman Prosedural”*, KK Rekayasa Perangkat Lunak dan Data, STEI, ITB, edisi April 2007 → digunakan di KU1071

Robert Sedgewick & Kevin Wayne, *“Algorithms”*, Addison-Wesley Professional, edisi ke-4, 2011. (booksite: <https://algs4.cs.princeton.edu/home/>)

Pustaka Tambahan

Jay Wengrow, “*A Common-Sense Guide to Data Structures and Algorithms*”, edisi ke-2

Brian W. Kernighan and Dennis M. Ritchie, “*The C Programming Language*”, edisi ke-2

Mark A. Weiss, “*Data Structures and Algorithm Analysis in C*”, edisi ke-2

N. Wirth, “*Algorithm and Data Structure*”

Jeri R. Hanly & Elliot B. Koffman, “*Problem Solving and Program Design in C*”

The GNU C Reference Manual.

Buku-buku/website/artikel relevan lain terkait struktur data dan bahasa C.

Beberapa Hal Penting

Tidak datang terlambat.

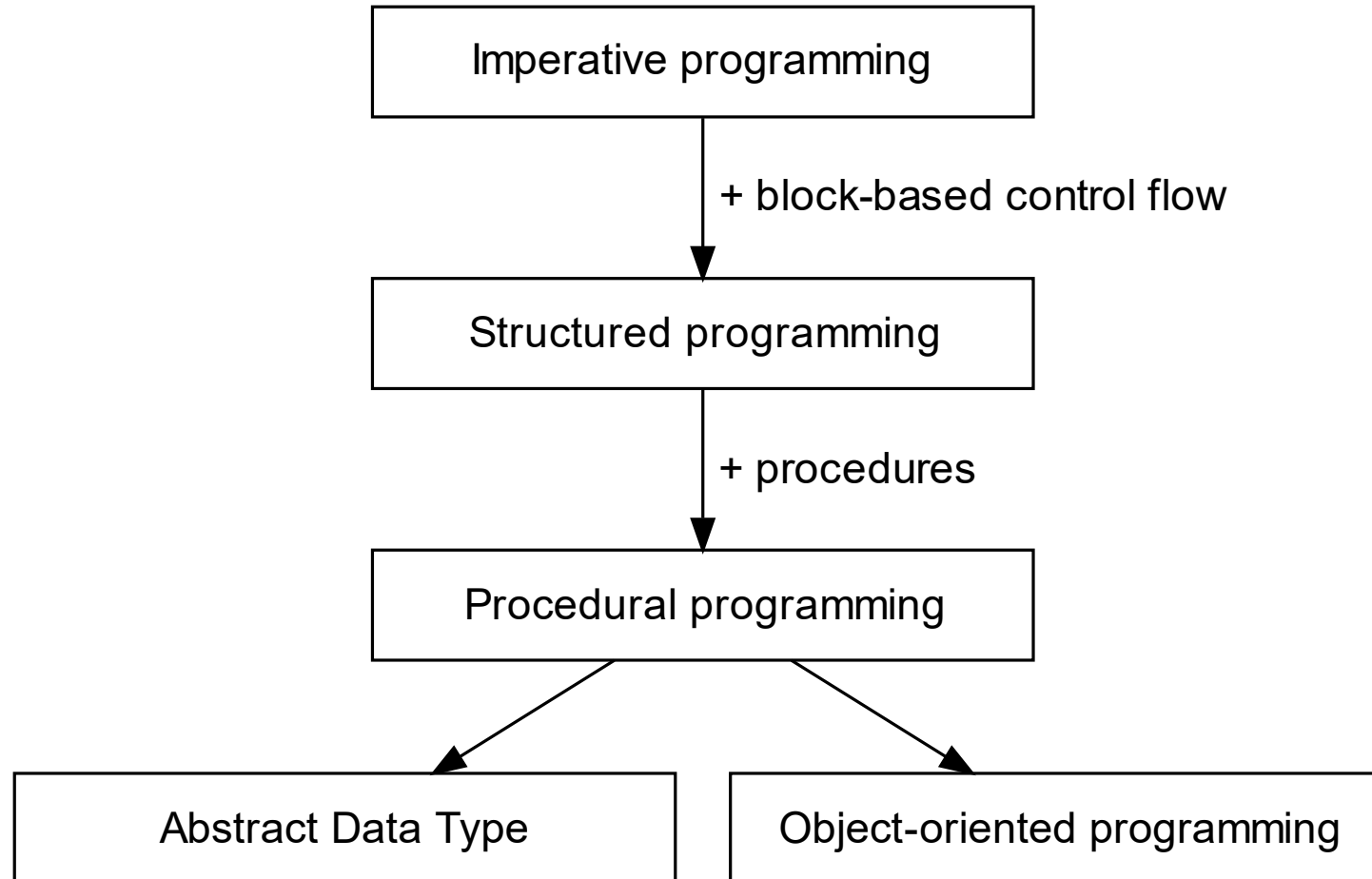
Jangan menunda ketidak-mengertian.

Pertanyaan?

Paradigma Prosedural

IF2110/IF2111 – Algoritma dan Struktur Data
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

Sejarah paradigma prosedural



Imperative programming

Paradigma mula-mula dalam pemrograman komputer.

Didasari pada cara kerja komputer yang mengeksekusi instruksi satu per satu secara berurutan.

Program pun ditulis sebagai suatu **untaian instruksi** berbentuk *statement*.

Merupakan paradigma memrogram dalam bahasa mesin ataupun *assembly*.

Menggunakan *statement* "goto" untuk alur kendali (*control flow*).

Contoh: loop pada *imperative programming*

KODE (C)	HASIL EKSEKUSI
<pre>#include <stdio.h> int main() { int i = 0; loop: printf("%d\n", i); if (++i < 5) goto loop; printf("Done!\n"); }</pre>	<pre>0 1 2 3 4 Done!</pre>

Structured programming

Statement goto dianggap berbahaya karena dapat membuat alur program lompat ke mana saja → sepotong kode dapat mengakses memori yang tidak valid:

```
int x = 0;  
goto there;  
int y = 1;  
there:  
printf("%d, %d\n", x, y);
```

Karena itu diperkenalkan ***control flow*** berbasis blok dengan lingkup (*scope*) memori yang terbatas pada setiap blok.

Kode di luar blok tidak dapat mengakses memori milik blok tersebut.

Contoh: lingkup sebuah blok

KODE (C)	HASIL EKSEKUSI
<pre>#include <stdio.h> int main() { int x = 5; if (x != 10) { int y = 8; printf("%d\n", y); // di sini `z` tidak dikenali } else { int z = 9; printf("%d\n", z); // di sini `y` tidak dikenali } // di sini `y` dan `z` tidak dikenali }</pre>	8

Procedural programming

Menambahkan *reusability* melalui subprogram berbentuk **prosedur** dan **fungsi**.

Prosedur: mengubah *state* program, tidak mengembalikan sebuah nilai.

Fungsi: melakukan pemetaan nilai (dalam parameter fungsi) ke nilai lain (sebagai *return value*). **Tidak** mengubah *state* program.

State program dalam konteks ini: nilai variabel di luar prosedur/fungsi.

Contoh: prosedur

KODE (C)	HASIL EKSEKUSI
<pre>#include <stdio.h> void swap(int *xp, int *yp) { int temp = *xp; *xp = *yp; *yp = temp; } int main() { int x = 5; int y = 10; printf("x=%d, y=%d\n", x, y); swap(&x, &y); printf("x=%d, y=%d\n", x, y); }</pre>	<pre>x=5, y=10 x=10, y=5</pre>

Contoh: fungsi

KODE (C)	HASIL EKSEKUSI
<pre>#include <stdio.h> int square(int x) { return x * x; } int main() { int a = 12; int sq = square(a); printf("%d² = %d\n", a, sq); }</pre>	12² = 144

+ Abstract data type (ADT)

Sebagaimana prosedur dan fungsi meningkatkan *reusability* pada aspek **algoritma**, ADT meningkatkan *reusability* pada aspek **struktur data**.

Dibahas lebih lanjut pada segmen berikutnya: **Algoritma, Struktur Data, dan ADT**.

Algoritma, Struktur Data, dan *Abstract Data Type* (ADT)

IF2110/IF2111 – Algoritma dan Struktur Data
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

Syarat

Dasar pemrograman (imperatif ~ prosedural):

Variabel, *assignment*

Array

Control flow:

Percabangan

Perulangan

Fungsi & Prosedur

Algoritma

Definisi umum: suatu prosedur, **langkah demi langkah**, untuk *menyelesaikan suatu masalah* atau *mencapai sebuah tujuan*.

Definisi khusus: suatu rangkaian **terhingga** dari instruksi-instruksi yang **terdefinisi** dan dapat diimplementasikan pada komputer untuk *menyelesaikan himpunan permasalahan spesifik* yang **computable**.

Terhingga: langkah-langkah harus berhenti *at some point*.

Terdefinisi: memenuhi semua klausa dari suatu definisi.

Computable: ada himpunan masalah yang belum diketahui apakah dapat diselesaikan dengan komputer atau tidak → bukan lingkup mata kuliah ini.

Contoh: algoritma Euclid untuk mencari FPB.

Struktur data

Adalah bagaimana data **diorganisasikan** dan **dikelola** agar dapat digunakan secara **efektif** dan **efisien**.

Contoh: merepresentasikan waktu (*time*) pukul 01:02:03

Cara I	Cara II
3 byte: h=01, m=02, s=03 0x01 0x02 0x03 • Menghitung selisih waktu lebih rumit • Mencetak waktu dalam format “HH:MM:SS” lebih mudah	2 byte: detik sejak tengah malam = 3723 0x0e 0x8b • Menghitung selisih waktu lebih mudah • Mencetak waktu dalam format “HH:MM:SS” lebih rumit

Dari contoh tersebut juga terlihat bahwa **struktur data yang berbeda** membutuhkan **algoritma yang berbeda** meskipun merepresentasikan **permasalahan yang sama**.

“Algoritma + Struktur Data = Program”

(Niklaus Wirth)

Abstract data type (ADT)

ADT adalah pemodelan suatu tipe data yang didefinisikan perilakunya berdasarkan:

data yang terkandung di dalamnya,

himpunan nilai yang mungkin dimiliki oleh data tersebut, serta

operasi yang dapat diterapkan terhadap data tersebut.

Implementasi dari suatu ADT mencakup **struktur data** untuk data yang didefinisikan oleh ADT dan **algoritma** untuk setiap operasi terhadap data tersebut.

ADT, struktur data, dan algoritma adalah salah satu mekanisme modularitas ($\rightarrow$ *reusability*) dalam paradigma prosedural.

Modularitas ADT, struktur data, dan algoritma

Menghitung selisih antara dua instan waktu, tanpa ADT. Bayangkan jika operasi ini perlu dilakukan berulang kali di program yang Anda buat:

```
{ waktu #1, 13:45:00 }      { waktu #2, 14:30:00 }
h1 ← 13                      ; h2 ← 14
m1 ← 45                      ; m2 ← 30
s1 ← 0                       ; s2 ← 0
{ selisih waktu #2 dengan waktu #1 }
ss1 ← h1*60*60 + m1*60 + s1 ; ss2 ← h2*60*60 + m2*60 + s2
d_ss ← ss2-ss1
```

Dengan ADT:

```
{ waktu #1, 13:45:00 }      { waktu #2, 14:30:00 }
CreateTime(t1, 13,45,0)      ; CreateTime(t2, 14,30,0)
d_t ← difference(t1,t2)
```

Modularitas tanpa ADT?

Arguably, beberapa hal dapat dimodularkan dengan fungsi/prosedur, tanpa ADT:

```
{ waktu #1, 13:45:00 }           { waktu #2, 14:30:00 }
h1 ← 13                          ; h2 ← 14
m1 ← 45                          ; m2 ← 30
s1 ← 0                           ; s2 ← 0
{ selisih waktu #2 dengan waktu #1 }
ss1 ← hhmmssToSeconds(h1,m1,s1) ; ss2 ← hhmmssToSeconds(h2,m2,s2)
d_ss ← ss2-ss1
{ atau: }
d_ss ← difference(h2,m2,s2,h1,m1,s1)
```

Namun mekanisme ini tidak mencegah pemrogram merangkai waktu secara salah, misalnya: `output(h1 + ":" + m2 + ":" + s1)` yang mencetak "13:30:00", yang secara logik tidak pernah relevan pada program di atas.

Ringkasan: ADT

Membuat suatu ADT mencakup:

Spesifikasi:

Definisi tipe data

Himpunan nilai yang mungkin

Daftar **operasi-operasi** (primitif maupun penunjang)

Prekondisi yang harus terpenuhi sebelum operasi dimulai

State setelah operasi selesai

Implementasi:

Struktur data konkret

Implementasi algoritma setiap operasi

“Test case” untuk setiap operasi pada setiap rentang kemungkinan nilai

Jenis struktur data & ADT umum

IF2110/IF2111 – Algoritma dan Struktur Data
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

Jenis struktur data umum

record atau ***tuple***

- Agregasi beberapa nilai dengan jumlah yang tetap.
- Setiap nilai dapat berbeda tipe.
- Elemennya diakses dari namanya.

array

- Sejumlah elemen yang diletakkan di memori secara kontigu.
- Setiap elemen bertipe data sama.
- Elemennya diakses melalui indeks.

node-based atau ***linked*** (struktur berkait)

- Sejumlah elemen yang saling terhubung melalui *pointer*.
- Setiap elemen bertipe data sama.
- Elemen diacu oleh elemen sebelumnya.

Record/tuple

Struktur data yang digunakan dalam ADT Time alt-1 adalah contoh **tuple**.

- Merupakan agregasi dari 3 nilai: **jam**, **menit**, dan **detik**.
- Masing-masing nilai dapat berasal dari tipe logik yang berbeda (**jam** hanya 0-23, sedangkan **menit** dan **detik** 0-59).
- Diakses berdasarkan namanya (misal: **t.hours**).

Dituliskan dengan tanda ‘<’ dan ‘>’, contoh:

Time dapat dituliskan sebagai **tuple** **<hours, minutes, seconds>**

Array

Array harus dialokasikan di memori dengan **ukuran yang sudah ditentukan**.

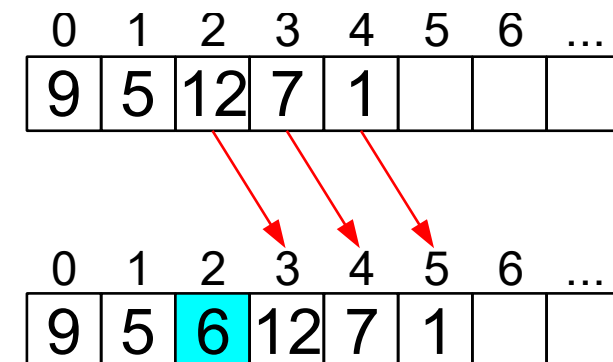
Misal: *array* 10 elemen bertipe integer → dialokasikan memori sebesar 10×4 byte.

Jika perlu mengubah ukuran *array*:

1. buat *array* baru dengan ukuran yang dikehendaki,
2. salin isi *array* lama ke *array* baru,
3. dealokasi *array* lama.

Akses ke setiap elemen *array* dilakukan melalui **indeks** (biasanya dimulai dari 0).

Menambah/menghapus elemen di tengah *array* mengakibatkan elemen-elemen setelah posisi tersebut harus **digeser satu per satu**.



Struktur berkait

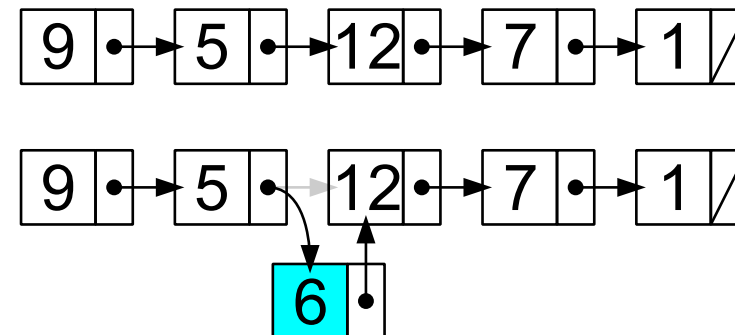
Sebuah *node* adalah **tuple** dengan dua elemen:

1. **Nilai** yang hendak disimpan dalam *node*,
2. **Pointer** ke *node* berikutnya.

Di memori, lokasi (alamat) *node* satu dan *node* berikutnya tidak harus bersebelahan.

Akses ke suatu *node* harus **melalui *node-node* sebelumnya**.

Menambah/menghapus elemen di tengah struktur berbasis *node* cukup dengan “**rerouting**” pointer-pointer.



Pakai yang mana?

Beberapa jenis data dapat distrukturkan dengan lebih dari satu cara. Contoh:

- Time bisa saja distrukturkan dalam **array** berukuran 3, sehingga bagian jam diakses dengan $t[0]$, menit dengan $t[1]$, dan detik dengan $t[2]$.
→ Konsekuensi: ketiga nilai tersebut kini bertipe sama (e.g., integer).
- Sebuah antrian (elemen yang baru datang harus “berdiri” di belakang, dan elemen yang boleh “dilayani” hanya elemen yang paling depan) dapat distrukturkan dalam bentuk **array** maupun **node-based**.

Pemilihan struktur data berakibat pada *trade-off* **efisiensi algoritma** pada berbagai operasi.

Diperlukan **analisis algoritma** untuk memilih struktur yang lebih efisien.

ADT umum yang akan dibahas

List/sequence

Matriks

Stack

Queue

Set,
Multiset

Map

Tree,
binary tree,
binary search tree

Graph

Notasi Algoritmik + Contoh ADT Sederhana

IF2110/IF2111 – Algoritma dan Struktur Data
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

Notasi algoritmik

Tidak mungkin mempelajari semua bahasa pemrograman → yang penting dipahami: **pola pikir komputasi** dan **paradigma pemrograman**.

Belajar memrogram $\neq$ Belajar bahasa pemrograman

Notasi Algoritmik: notasi standar yang digunakan untuk menuliskan teks algoritma [dalam paradigma prosedural].

Algoritma adalah solusi detail secara prosedural dari suatu persoalan dalam Notasi Algoritmik.

Program adalah program komputer dalam suatu bahasa pemrograman yang tersedia di dunia nyata.

Perlunya Notasi Algoritmik

Notasi Algoritmik → representasi cara berpikir untuk menyelesaikan persoalan dengan paradigma prosedural.

Membuat program → melihat “kosa kata” translasi notasi algoritmik ke bahasa pemrograman yang dipilih.

Dengan notasi algoritmik yang sama, bisa dibuat program dalam bahasa pemrograman yang berbeda, dengan paradigma pemrograman yang sama.

Contoh: untuk prosedural: C, Pascal, Fortran.

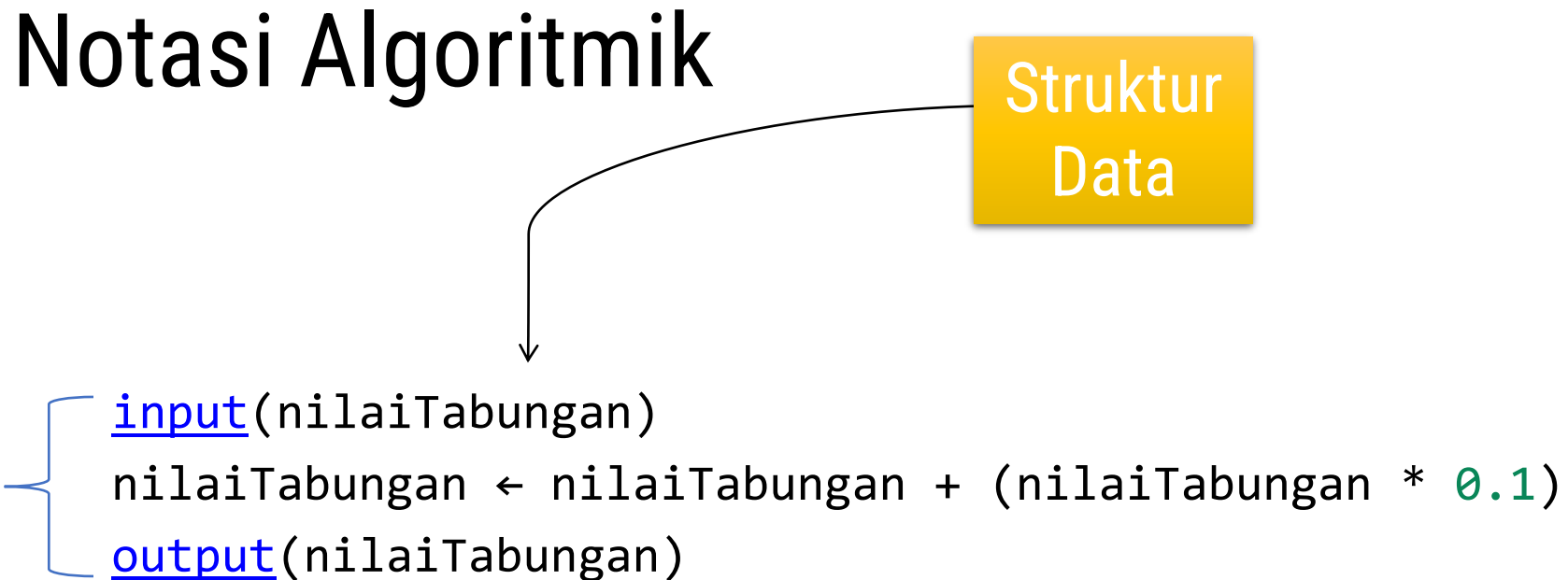
Contoh Kasus: Tabungan

Program = Algoritma + Struktur Data

Notasi Algoritmik

Struktur
Data

Algoritma



```
input(nilaiTabungan)  
nilaiTabungan <- nilaiTabungan + (nilaiTabungan * 0.1)  
output(nilaiTabungan)
```


Kode Program Bahasa C

```
input(nilaiTabungan)  
nilaiTabungan ← nilaiTabungan + (nilaiTabungan * 0.1)  
output(nilaiTabungan)
```



scanf akan membaca
dari hasil ketik di
keyboard

→ `scanf("%f", &nilaiTabungan);`

`nilaiTabungan += nilaiTabungan * 0.1;`

printf akan menulis hasil
di layar komputer

→ `printf("%f\n", nilaiTabungan);`

Kode Program Bahasa Pascal

```
input(nilaiTabungan)  
nilaiTabungan ← nilaiTabungan + (nilaiTabungan * 0.1)  
output(nilaiTabungan)
```



readln akan membaca
dari hasil ketik di
keyboard

→ `readln(nilaiTabungan);`

```
nilaiTabungan := nilaiTabungan +  
                nilaiTabungan * 0.1;
```

writeln akan menulis
hasil di layar komputer

→ `writeln(nilaiTabungan);`

Kode Program Bahasa Python

```
input(nilaiTabungan)  
nilaiTabungan ← nilaiTabungan + (nilaiTabungan * 0.1)  
output(nilaiTabungan)
```



input akan membaca
dari hasil ketik di
keyboard

—————→ `nilaiTabungan = float(input())`

`nilaiTabungan += nilaiTabungan * 0.1`

print akan menulis hasil
di layar komputer

—————→ `print(nilaiTabungan)`

Struktur program notasi algoritmik dalam contoh

Program Penjumlahan

{ Spesifikasi Program: menghitung $a + b$ }

KAMUS

{ Deklarasi variabel }

a, b: integer

ALGORITMA

input(a)

input(b)

$a \leftarrow a + b$

output(a)

Contoh spesifikasi ADT dengan notasi algoritmik

Struktur data

{ Modul ADT Time }

```
type Time: < hours: integer[0..23],  
               minutes: integer[0..59],  
               seconds: integer[0..59] >
```

Possible values

```
{ 0 ≤ hours ≤ 23 }  
{ 0 ≤ minutes ≤ 59 }  
{ 0 ≤ seconds ≤ 59 }
```

Deklarasi operasi

{ Konstruktor: membentuk Time dari komponen-komponennya: h sebagai hours, m sebagai minutes, dan s sebagai seconds. }

```
procedure CreateTime(output t: Time, input h: integer[0..23],  
                    input m: integer[0..59], input s: integer[0..59])
```

{ Mendapatkan komponen hours dari T }

```
function getHours(T: Time) → integer[0..23]
```

{ Mendapatkan komponen minutes dari T }

```
function getMinutes(T: Time) → integer[0..59]
```

{ Mendapatkan komponen seconds dari T }

```
function getSeconds(T: Time) → integer[0..59]
```

{ Selisih antara dua Time, dalam satuan detik }

```
function difference(start: Time, end: Time) → integer
```

Contoh realisasi operasi dalam notasi algoritmik

Signature operasi

Prasvarat operasi

Ekspektasi hasil operasi pada setiap rentang kemungkinan masukan (*test case*)

`(start, end: Time) → integer`

`time start dan end, dengan syarat  $start \leq end$ . }`

{ EXPECT

`CreateTime(start, 1,2,3); CreateTime(end, 2,3,4)`

`⇒ difference(start,end) = 3661`

`CreateTime(start, 2,3,4); CreateTime(end, 2,3,4)`

`⇒ difference(start,end) = 0`

`CreateTime(start, 2,3,4); CreateTime(end, 1,2,3)`

`⇒ difference(start,end) = tak terdefinisi }`

Body operasi

KAMUS LOKAL

`startSec, endSec: integer`

ALGORITMA

`startSec ← getHours(start)*60*60 + getMinutes(start)*60 + getSeconds(start)`

`endSec ← getHours(end)*60*60 + getMinutes(end)*60 + getSeconds(end)`

`→ endSec-startSec`

*Spesifikasi lengkap notasi algoritmik terdapat pada dokumen terpisah.

Operasi primitif dan penunjang

Operasi primitif

Beberapa operasi sangat bergantung pada struktur data yang digunakan.

Operasi-operasi tersebut adalah operasi primitif.

Pada contoh ADT Time sebelumnya, `CreateTime`, `getHours`, `getMinutes`, dan `getSeconds` akan memiliki implementasi yang berbeda jika Time menggunakan representasi detik saja.

Sementara itu, operasi selisih tidak harus bergantung pada struktur data karena implementasinya dapat memanfaatkan primitif yang sudah ada.

(Tidak mempertimbangkan efisiensi algoritma.)

Contoh: ADT Time alt-1 dan CreateTime

```
{ Modul ADT Time alt-1 }
```

```
type Time: < hours: integer[0..23],    { 0 ≤ hours ≤ 23 }  
             minutes: integer[0..59],    { 0 ≤ minutes ≤ 59 }  
             seconds: integer[0..59] > { 0 ≤ seconds ≤ 59 }
```

```
{ Konstruktor: membentuk Time t dari komponen-komponennya: h sebagai hours, m sebagai  
  minutes, dan s sebagai seconds. }
```

```
procedure CreateTime(output t: Time, input h: integer[0..23],  
                    input m: integer[0..59], input s: integer[0..59])
```

KAMUS

-

ALGORITMA

```
t.hours ← h  
t.minutes ← m  
t.seconds ← s
```

Contoh: ADT Time alt-2 dan CreateTime

```
{ Modul ADT Time alt-2 }
```

```
type Time: < seconds: integer[0..86399] > {  $0 \leq \text{seconds} \leq 86400$  }
```

```
{ Konstruktor: membentuk Time t dari komponen-komponennya: h sebagai hours, m sebagai minutes, dan s sebagai seconds. }
```

```
procedure CreateTime(output t: Time, input h: integer[0..23],  
                    input m: integer[0..59], input s: integer[0..59])
```

KAMUS

-

ALGORITMA

```
t.seconds  $\leftarrow$  h*60*60 + m*60 + s
```

Contoh: implementasi getHours(t)

```
{ alt-1 }  
{ Mendapatkan bagian hours dari t }  
function getHours(t: Time) → integer[0..23]
```

KAMUS

-

ALGORITMA

→ t.hours

```
{ alt-2 }  
{ Mendapatkan bagian hours dari t }  
function getHours(t: Time) → integer[0..23]
```

KAMUS

-

ALGORITMA

→ t.seconds **div** (60*60)

Contoh: selisih dua waktu

function difference(start: Time, end: Time) → integer
{ Menghasilkan selisih antara dua Time start dan end, dengan syarat $\text{start} \leq \text{end}$. }

{ **EXPECT**

CreateTime(start, 1,2,3); CreateTime(end, 2,3,4)

⇒ difference(start,end) = 3661

CreateTime(start, 2,3,4); CreateTime(end, 2,3,4)

⇒ difference(start,end) = 0

CreateTime(start, 2,3,4); CreateTime(end, 1,2,3)

⇒ difference(start,end) = **tak terdefinisi** }

KAMUS LOKAL

startSec, endSec: integer

ALGORITMA

startSec ← getHours(start)*60*60 + getMinutes(start)*60 + getSeconds(start)

endSec ← getHours(end)*60*60 + getMinutes(end)*60 + getSeconds(end)

→ endSec-startSec

Tugas

Ambil sebuah program yang pernah Anda tulis pada kuliah **Dasar Pemrograman** bagian pemrograman prosedural, yang minimal memiliki:

Percabangan

Perulangan

Prosedur/fungsi

Tulis kembali program tersebut dalam **Notasi Algoritmik**.

Gunakan diktat kuliah sebagai referensi Notasi Algoritmik.

Tujuan: mengenali dan membiasakan diri menulis program dalam Notasi Algoritmik.

Bahasa C

IF2110/IF2111 – Algoritma dan Struktur Data
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

Bahasa C – Sejarah singkat

Dikembangkan oleh Dennis Ritchie dan Brian Kernighan pada awal 1970an.

Awalnya berkembang di lingkungan Unix

±90% sistem operasi Unix ditulis dalam bahasa C

Standar Bahasa C yang ada

- Definisi Kernighan dan Ritchie (K&R)
- ANSI-C → dipakai dalam kuliah ini
- Definisi AT&T (untuk C++)

Versi C pada sistem operasi Windows:

Lattice C, Microsoft C, Turbo C, dll.

Bahasa C – Kegunaan

Bahasa C banyak digunakan untuk:

- Membuat sistem operasi dan program-program sistem,
- Pemrograman yang dekat dengan perangkat keras (misal: kontrol peralatan),
- Membuat *toolkit*,
- Menulis program aplikasi (misalnya dBase, Wordstar, Lotus123).

Kelebihan Bahasa C:

- Kode yang *compact* & efisien.

Kekurangan Bahasa C:

- Kurang *readable* dibandingkan bahasa tingkat tinggi lain.

Bahasa C – Beberapa catatan

Blok Program

Sekumpulan kalimat (*statement*) C di antara kurung kurawal { dan }

Contoh:

```
if (a > b) {  
    printf("a lebih besar dari b\n");  
    b += a;  
    printf("sekarang b lebih besar dari a\n");  
}
```

Komentar program

Dituliskan di antara tanda /* dan */

Alternatif: // hingga *end of line* (biasanya \n)

Bahasa C adalah bahasa yang ***case-sensitive***

Translasi: notasi algoritmik ke bahasa C

```
{ Modul ADT Time }  
type Time: < hours: integer[0..23],      { 0 ≤ hours ≤ 23 }  
             minutes: integer[0..59],      { 0 ≤ minutes ≤ 59 }  
             seconds: integer[0..59] > { 0 ≤ seconds ≤ 59 }
```

{ Konstruktor: membentuk Time dari komponen-komponennya: h sebagai hours, m sebagai minutes, dan s sebagai seconds. }

```
procedure CreateTime(output t: Time, input h: integer[0..23],  
                    input m: integer[0..59], input s: integer[0..59])
```

{ Mendapatkan komponen hours dari T }

```
function getHours(T: Time) → integer[0..23]
```

{ Mendapatkan komponen minutes dari T }

```
function getMinutes(T: Time) → integer[0..59]
```

{ Mendapatkan komponen seconds dari T }

```
function getSeconds(T: Time) → integer[0..59]
```

{ Selisih antara dua Time, dalam satuan detik }

```
function difference(start: Time, end: Time) → integer
```

Translasi: notasi algoritmik ke bahasa C

File: src/time.h

```
/* Modul ADT Time */
#ifndef TIME_H
#define TIME_H

typedef struct Time { int hours;
                     int minutes;
                     int seconds; } time;

/* (Komentar-komentar tidak dituliskan untuk mempersingkat.) */
void CreateTime(time &t, int h, int m, int s);
int getHours(time t);
int getMinutes(time t);
int getSeconds(time t);
int difference(time start, time end);

#endif /* TIME_H */
```

Translasi: notasi algoritmik ke bahasa C

function difference(start: Time, end: Time) → integer
{ Menghasilkan selisih antara dua Time start dan end, dengan syarat $\text{start} \leq \text{end}$. }

{ **EXPECT**

CreateTime(start, 1,2,3); CreateTime(end, 2,3,4)

⇒ difference(start,end) = 3661

CreateTime(start, 2,3,4); CreateTime(end, 2,3,4)

⇒ difference(start,end) = 0

CreateTime(start, 2,3,4); CreateTime(end, 1,2,3)

⇒ difference(start,end) = **tak terdefinisi** }

KAMUS LOKAL

startSec, endSec: integer

ALGORITMA

startSec ← getHours(start)*60*60 + getMinutes(start)*60 + getSeconds(start)

endSec ← getHours(end)*60*60 + getMinutes(end)*60 + getSeconds(end)

→ endSec-startSec

Translasi: notasi algoritmik ke bahasa C

File: src/time.c

```
#include "time.h"
```

```
int difference(time start, time end) {  
    /* Menghasilkan selisih antara dua Time start dan end, dengan syarat  
       start ≤ end. */  
    /* KAMUS LOKAL */  
    int startSec, endSec;  
    /* ALGORITMA */  
    startSec = getHours(start)*60*60 + getMinutes(start)*60 + getSeconds(start);  
    endSec   = getHours(end)*60*60   + getMinutes(end)*60   + getSeconds(end);  
    return endSec - startSec;  
}
```

Translasi: notasi algoritmik ke bahasa C

File: tests/check_time.c

```
#include <check.h>
```

```
#include "../src/time.h"
```

```
time t1, t2;
```

```
void setup() { CreateTime(t1, 2,3,4); CreateTime(t2, 1,2,3); }
```

```
START_TEST(time_difference) {  
    ck_assert_int_eq(difference(t1,t2), 3661);  
    ck_assert_int_eq(difference(t1,t1), 0);  
} END_TEST
```

Berikutnya: sintaks Bahasa C

Type, konstanta, variabel, assignment

Input/output

Analisis kasus

Pengulangan

Subprogram

Type, Konstanta, Variabel, Assignment

Konstanta, variabel

Notasi Algoritmik

Konstanta

constant <nama>:<type>=<harga>

Deklarasi variabel

<nama>: <type>

Inisialisasi/assignment

<nama> ← <harga>

Bahasa C

// Konstanta

// 1) Dengan const:

const [<type>] <nama> = <harga>;

// 2) Dengan preprocessor/macro

#define <nama> <harga>

// Deklarasi Variabel

<type> <nama>;

// Inisialisasi/assignment

<nama> = <harga>;

// Deklarasi sekaligus inisialisasi

<type> <nama> = <harga>;

Konstanta, variabel: contoh

Notasi Algoritmik

Program Test

{Tes konstanta, variabel, assignment, inisialisasi...}

KAMUS

{Konstanta}

constant LIMA: real = 5.0

constant PI: real = 3.14

{Variabel}

luas, r: real

i : integer

ALGORITMA

i ← 1 {Inisialisasi}

r ← 10.0 {Inisialisasi}

...

Bahasa C

```
/* File: test.c - Program Test */
/* Tes konstanta, variabel, assignment,
inisialisasi... */
```

```
/* KAMUS */
```

```
#define LIMA 5.0 // deklarasi konstanta cara-1
```

```
int main () {
```

```
    /* KAMUS */
```

```
    /* Konstanta */
```

```
    const float PI = 3.14; // dekl. konstanta cara-2
```

```
    /* Variabel */
```

```
    float luas, r;
```

```
    int i = 1; /* Deklarasi + inisialisasi */
```

```
    /* ALGORITMA */
```

```
    r = 10.0; /* Inisialisasi */
```

```
    ...
```

```
    return 0;
```

```
}
```

Catatan: C preprocessor

Pada contoh sebelumnya, konstanta LIMA dideklarasikan menggunakan **macro**

```
#define LIMA 5.0
```

Compiler terlebih dahulu akan menjalankan *preprocessor* sebelum kompilasi dilakukan.

Salah satu tugas *preprocessor* adalah mengganti kemunculan *macro* sesuai deklarasinya,

Pada contoh tadi, semua kemunculan LIMA pada *source code* akan di-*replace* dengan 5.0 sebelum dilakukan kompilasi.

Assignment

Notasi Algoritmik

Assignment

```
<nama1> ← <nama2>
<nama> ← <konstanta>
<nama> ← <ekspresi>
nama1 ← nama1 <opr> nama2
```

Contoh:

```
luas ← PI * r * r
x ← x * y
i ← i + 1

i ← i - 1
```

Bahasa C

// Assignment

```
<nama1> = <nama2>;
<nama> = <konstanta>;
<nama> = <ekspresi>;
nama1 = nama1 <opr> nama2;
// Compound Assignment:
nama1 <opr>= nama2;
```

// Contoh:

```
luas = PI * r * r;
x *= y;
i++;
++i; /* Apa bedanya? */
i--;
--i; /* Apa bedanya? */
```

Type data karakter (Bahasa C)

Contoh deklarasi:

```
char cc;
```

Contoh literal:

'c' → karakter *c*

'0' → karakter *0*

'\013' → karakter vertical tab

Jenis-jenis character:

[signed] char

unsigned char

char pada dasarnya adalah integer 1 byte (8 bits)

Type data bil. bulat (Bahasa C)

Contoh deklarasi: `int i; short int j;`

Contoh literal: `1 2 0 -1`

Jenis-jenis integer:

`[signed] int`

Natural size of integers in host machine, e.g. 32 bits

No shorter than short int, no longer than long int

`[signed] short [int]`

Min. 16 bits of integer

`[signed] long [int]`

Min. 32 bits of integer

`unsigned int, unsigned short [int], unsigned long [int]`

0 and positive integers only.

Type data bil. real (Bahasa C)

Contoh deklarasi: `float f1; double f2;`

Contoh literal: `3.14 0.0 1.0e+2 5.3e-2`

Jenis-jenis real:

`float`

Single-precision floating point

6 digits decimal

`double`

Double-precision floating point

Eg. 10 digits decimal

`long double`

Extended-precision floating point

Eg. 18 digits decimal

Type data boolean (Bahasa C)

C tidak menyediakan type boolean

Ada banyak cara untuk mendefinisikan boolean

Cara 1 – Digunakan nilai integer untuk menggantikan nilai *true* & *false*:

true = nilai bukan 0

false = 0

Cara 2 – Definisikan sebagai konstanta:

```
#define boolean unsigned char
#define TRUE 1
#define FALSE 0
```

Type data boolean (Bahasa C)

Cara 3 – Definisikan dalam file *header*,
misal: `boolean.h` → digunakan sebagai
standar dalam mata kuliah ini

```
/* File: boolean.h */
/* Definisi type data boolean */

#ifndef BOOLEAN_H
#define BOOLEAN_H

#define boolean unsigned char
#define TRUE 1
#define FALSE 0

#endif
```

Contoh penggunaan:

```
/* File: boolean_test.c */
#include "boolean.h"

int main () {
    /* KAMUS */
    boolean found;
    ...

    /* ALGORITMA */
    found = TRUE;
    ...

    return 0;
}
```

Type data string (Bahasa C)

Dalam C, string adalah *pointer* ke *array* dengan elemen char

Contoh konstanta string: "Ini sebuah string"

Konstanta string berada di antara *double quotes* "..."

Namun, string $\neq$ array of char

Representasi internal string selalu diakhiri character ' $\backslash 0$ ', sedangkan array of character tidak

Jadi, string "A" sebenarnya terdiri atas dua buah character yaitu 'A' dan ' $\backslash 0$ '

Type data string (Bahasa C)

Contoh deklarasi (+ inisialisasi):

```
char str1[] = "ini string"; /* deklarasi dan inisialisasi */  
char str2[37]; /* deklarasi string sepanjang 37 karakter */  
char *str3;
```

Contoh cara assignment nilai:

```
strcpy(str2, "pesan apa"); /* str2 diisi dgn "pesan apa" */  
  
str3 = (char *) malloc (20 * sizeof(char)); /* alokasi memori terlebih dahulu */  
  
strcpy(str3, ""); /* str3 diisi dengan string kosong */  
  
/* HATI-HATI, cara di bawah ini SALAH! */  
str3 = "Kamu pesan apa";
```

Type enumerasi

Notasi Algoritmik

KAMUS

```
{ Definisi type }  
type Hari: (senin, selasa, rabu, Kamis,  
            jumat, sabtu, minggu)
```

```
{ Deklarasi variabel }  
h: Hari
```

ALGORITMA

```
{ Assignment }  
h ← senin
```

Bahasa C

```
/* KAMUS */  
/* Definisi type */  
typedef enum {  
    senin, selasa, rabu, Kamis,  
    jumat, sabtu, minggu  
} hari; /* senin=0, selasa=1, dst. */  
  
int main() {  
    /* Deklarasi variabel */  
    hari h;  
  
    /* ALGORITMA */  
    /* Assignment */  
    h = senin; /* h = 0 */  
}
```

Type bentukan (tuple)

Notasi Algoritmik

KAMUS

```
{ Definisi Type }  
type namatype:  
  < elemen1: type1,  
    elemen2: type2,  
    ... >
```

```
{ Deklarasi Variabel }  
nmvar1: namatype  
nmvar2: type1 {misal}
```

ALGORITMA

```
{ Akses Elemen }  
nmvar2 ← nmvar1.elemen1  
nmvar1.elemen2 ← <ekspresi>
```

Bahasa C

```
/* KAMUS */  
/* Definisi Type */  
typedef struct NamaType {  
    type1 elemen1;  
    type2 elemen2;  
    ...  
} namatype;  
  
int main() {  
    /* Deklarasi Variabel */  
    namatype nmvar1;  
    type1 nmvar2; /*misal*/  
  
    /* ALGORITMA */  
    /* Akses Elemen */  
    nmvar2 = nmvar1.elemen1;  
    nmvar1.elemen2 = <ekspresi>;  
}
```

Type bentukan (contoh)

Notasi Algoritmik

KAMUS

```
{ Definisi Type }  
type point:  
  < x: integer,  
    y: integer >
```

```
{ Deklarasi Variabel }  
p: point  
bil: integer {misal}
```

ALGORITMA

```
{ Akses Elemen }  
bil ← p.x  
p.y ← 20
```

Bahasa C

```
/* KAMUS */  
/* Definisi Type */  
typedef struct Point {  
    int x;  
    int y;  
} point;  
  
int main() {  
    /* Deklarasi Variabel */  
    point p;  
    int bil; /* misal */  
  
    /* ALGORITMA */  
    /* Akses Elemen */  
    bil = p.x;  
    p.y = 20;  
}
```

Operator

Notasi algoritmik

Ekspresi Infix:

<opn1> <opr> <opn2>

Contoh: $x + 1$

Operator Numerik:

+

-

*

/

div

mod

Bahasa C

Ekspresi Infix:

<opn1> <opr> <opn2>

Contoh: $x + 1$

Operator Numerik:

+

-

*

/ */*hasil float*/*

/ */*hasil integer*/*

%

++ */*increment*/*

-- */*decrement*/*

Operator

Notasi algoritmik

Operator Relasional:

>	<
≥	≤
=	≠

Operator Logika:

AND
OR
NOT

Bahasa C

Operator Relasional:

>	<
>=	<=
==	!=

Operator Logika:

&&
||
!

Operator Bit:

```
<<  /*shift left*/  
>>  /*shift right*/  
&    /*and*/  
|    /*or*/  
^    /*xor*/  
~    /*not*/  
/*Lihat contoh di diktat*/
```

Input/Output

Input

Notasi Algoritmik

input(<list-nama>)

Contoh:

input(x) {x integer}

input(x, y)

input(f) {f real}

input(s) {s string}

Bahasa C

```
scanf("<format>", <list-nama>);
```

// Contoh:

```
scanf("%d",&x); /*x integer*/
```

```
scanf("%d %d", &x, &y);
```

```
scanf("%f",&f); /*f real*/
```

```
scanf("%s",s); /*s string*/
```

// format-format sederhana:

%d untuk type integer

%f untuk type real

%c untuk type character

%s untuk type string

Output

Notasi Algoritmik

output(<list-nama>)

Contoh:

output("Nomor: ", x, " dapat nilai ", y)
 {x, y integer}

output(x, y)
output("Contoh output")
output("Namaku: ", nama)

output(f) {f real}
output(cc) {cc character}

Bahasa C

printf("<format>", <list-nama>);

// Contoh:

printf("Nomor: %d dapat nilai %d", x, y);
 /* x, y integer */

printf("%d %d", x, y);
printf("Contoh output");
printf("Namaku: %s", nama);

printf("%f", f); /* f real */
printf("%c", cc); /* cc character */

/* Format-format sederhana sama seperti pada
input */

Analisis Kasus

Analisis Kasus

Notasi Algoritmik

Satu Kasus:

```
if kondisi then  
    aksi
```

Dua Kasus Komplementer:

```
if kondisi-1 then  
    aksi-1  
else { not kondisi-1 }  
    aksi-2
```

Bahasa C

// Satu Kasus:

```
if (kondisi) {  
    aksi;  
}
```

// Dua Kasus Komplementer:

```
if (kondisi-1) {  
    aksi-1;  
} else { /* not kondisi-1 */  
    aksi-2;  
}
```

Analisis Kasus (> 2 kasus)

Notasi Algoritmik

depend on nama

kondisi-1: aksi-1

kondisi-2: aksi-2

...

kondisi-n: aksi-n

depend on nama

kondisi-1: aksi-1

kondisi-2: aksi-2

...

else : aksi-else

Bahasa C

```
if (kondisi-1) {  
    aksi-1;  
} else if (kondisi-2) {  
    aksi-2;  
}  
...  
} else if (kondisi-n) {  
    aksi-n;  
}  
}
```

```
if (kondisi-1) {  
    aksi-1;  
} else if (kondisi-2) {  
    aksi-2;  
}  
...  
} else { aksi-else; }
```

Analisis Kasus (> 2 kasus)

Jika setiap kondisi dapat dinyatakan dalam bentuk:
nama = const-exp (const-exp adalah suatu ekspresi konstan),
maka dapat digunakan *statement* **switch**.

Notasi Algoritmik

```
depend on nama  
  nama=const-exp-1: aksi-1  
  nama=const-exp-2: aksi-2  
  ...  
  else           : aksi-else
```

Bahasa C

```
switch (nama) {  
  case const-exp-1:  
    aksi-1;  
    [break;]  
  case const-exp-2:  
    aksi-2;  
    [break;]  
  ...  
  default:  
    aksi-else;  
    [break;]  
};
```


Contoh

Diktat “Contoh Program Kecil Bahasa C”

Instruksi Kondisional

Pengulangan

Pengulangan

Notasi Algoritmik

Pengulangan berdasarkan kondisi berhenti:

```
repeat  
    Aksi  
until kondisi-stop
```

Pengulangan berdasarkan kondisi ulang:

```
while (kondisi-ulang) do  
    Aksi  
{not kondisi-ulang}
```

Bahasa C

```
do {  
    Aksi;  
} while (!kondisi-stop);
```

```
while (kondisi-ulang) {  
    Aksi;  
} /*not kondisi-ulang */
```

Pengulangan

Notasi Algoritmik

Pengulangan berdasarkan pencacah:

i traversal [Awal..Akhir]
 Aksi

Bahasa C

```
/* Jika Awal <= Akhir */  
for(i=Awal;i<=Akhir;i++) {  
    Aksi;  
}  
/* Jika Awal >= Akhir */  
for(i=Awal;i>=Akhir;i--) {  
    Aksi;  
}
```

Catatan:

```
for(exp1;exp2;exp3) {  
    Aksi;  
}  
ekivalen dengan:  
exp1;  
while (exp2) {  
    Aksi;  
    exp3;  
} /* !exp2 */
```

Pengulangan

Notasi Algoritmik

Pengulangan berdasarkan dua aksi:

iterate

Aksi-1

stop kondisi-stop

Aksi-2

Bahasa C

```
for(;;) {  
    Aksi-1;  
    if (kondisi-stop)  
        break;  
    else  
        Aksi-2;  
}
```

Contoh

Diktat “Contoh Program Kecil Bahasa C”

Pengulangan

Subprogram

Fungsi (Notasi algoritmik)

<pre><u>function</u> NMAF (param1: type1, param2: type2, ...) → type-hasil { Spesifikasi fungsi }</pre>
KAMUS LOKAL <pre>{ Semua nama yang dipakai dalam algoritma dari fungsi }</pre>
ALGORITMA <pre>{ Deretan fungsi algoritmik: pemberian harga, input, output, analisis kasus, pengulangan } { Pengiriman harga di akhir fungsi, harus sesuai dengan type-hasil } → hasil</pre>

Fungsi (Bahasa C)

```
type-hasil NAMA_F (type1 param1, type2 param2, ...) {  
    /* Spesifikasi fungsi */  
  
    /* KAMUS LOKAL */  
    /* Semua nama yang dipakai dalam algoritma dari  
       fungsi */  
  
    /* ALGORITMA */  
    /* Deretan fungsi algoritmik:  
       pemberian harga, input, output, analisis kasus,  
       pengulangan */  
  
    /* Pengiriman harga di akhir fungsi, harus sesuai  
       dengan type-hasil */  
    return hasil;  
}
```

Pemanggilan fungsi (Notasi algo.)

Program POKOKPERSOALAN

{ Spesifikasi: Input, Proses, Output }

KAMUS

{ semua nama yang dipakai dalam algoritma }

{ Prototype fungsi }

function NMAF ([list nama:type input]) → **type-hasil**

{ Spesifikasi fungsi }

ALGORITMA

{ Deretan instruksi pemberian harga, input, output, analisa kasus, pengulangan yang memakai fungsi }

{ Harga yang dihasilkan fungsi juga dapat dipakai dalam ekspresi }

nama ← NMAF ([list parameter aktual])

output (NMAF ([list parameter aktual]))

{ Body/Realisasi Fungsi diletakkan setelah program utama }

Pemanggilan fungsi (Bahasa C)

```
/* Program POKOKPERSOALAN */
/* Spesifikasi: Input, Proses, Output */
/* Prototype Fungsi */
type-hasil NMAF ([list <type nama> input]);
/* Spesifikasi Fungsi */
int main () {
    /* KAMUS */
    /* Semua nama yang dipakai dalam algoritma */
    /* ALGORITMA */
    /* Deretan instruksi pemberian harga, input, output, analisis
        kasus, pengulangan yang memakai fungsi */
    /* Harga yang dihasilkan fungsi juga dapat dipakai dalam
        ekspresi */
    nama = NMAF ([list parameter aktual]);
    printf ("[format]", NMAF ([list parameter aktual]));
    /* Harga yang dihasilkan fungsi juga dapat dipakai dalam
        ekspresi */

    return 0;
}
/* Body/Realisasi Fungsi diletakkan setelah program utama */
```

Prosedur (Notasi algoritmik)

```
procedure NAMAP (input nama1: type1,  
                  input/output nama2: type2,  
                  output nama3: type3)  
{ Spesifikasi, Initial State, Final State }
```

KAMUS LOKAL

```
{ Semua nama yang dipakai dalam BADAN PROSEDUR }
```

ALGORITMA

```
{ BADAN PROSEDUR }  
{ Deretan instruksi pemberian harga, input, output,  
  analisis kasus, pengulangan atau prosedur }
```

Prosedur (Bahasa C)

```
void NAMAP (type1 nama1, type2 *nama2, type3 *nama3)
/* Spesifikasi, Initial State, Final State */
{
    /* KAMUS LOKAL */
    /* Semua nama yang dipakai dalam BADAN PROSEDUR */

    /* ALGORITMA */
    /* Deretan instruksi pemberian harga, input, output,
       analisis kasus, pengulangan atau prosedur */

}
```

Pemanggilan Prosedur (Notasi alg.)

Program POKOKPERSOALAN

{ Spesifikasi: Input, Proses, Output }

KAMUS

{semua nama yang dipakai dalam algoritma }

{Prototype prosedur}

procedure NAMAP (input nama1: type1,
 input/output nama2: type2,
 output nama3: type3)

{Spesifikasi prosedur, initial state, final state}

ALGORITMA

{Deretan instruksi pemberian harga, input, output, analisis kasus,
pengulangan}

NAMAP(paramaktual1, paramaktual2, paramaktual3)

{Body/Realisasi Prosedur diletakkan setelah program utama}

Pemanggilan Prosedur (Bahasa C)

```
/* Program POKOKPERSOALAN */
/* Spesifikasi: Input, Proses, Output */

/* Prototype prosedur */
void NAMAP (type1 nama1, type2 *nama2, type3 *nama3);
/* Spesifikasi prosedur, initial state, final state */

int main () {
    /* KAMUS */
    /* semua nama yang dipakai dalam algoritma */

    /* ALGORITMA */
    /* Deretan instruksi pemberian harga, input, output,
       analisis kasus, pengulangan */
    NAMAP(paramaktual1, &paramaktual2, &paramaktual3);

    return 0;
}

/* Body/Realisasi Prosedur diletakkan setelah program utama */
```

Modularitas Program dalam Bahasa C

IF2110/IF2111 – Algoritma dan Struktur Data
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

Tujuan

Mahasiswa memahami kegunaan modul program

Mahasiswa memahami konsep reusability dalam pembuatan program

Mahasiswa memahami pembuatan program C dengan beberapa modul program (dalam beberapa file)

Mahasiswa dapat mengimplementasikan program dengan memakai modul program dalam bahasa C

Modularitas Program

Sebuah program yang “utuh”, seringkali terdiri dari beberapa modul program.

Modul program dapat mewakili:

- Sekumpulan **rutin** (prosedur & fungsi) sejenis
- **ADT** (Abstract Data Type): definisi type dan primitifnya
- **Mesin** : definisi *state variable* dari mesin dan primitifnya

Pembuatan program

Program terdiri dari :

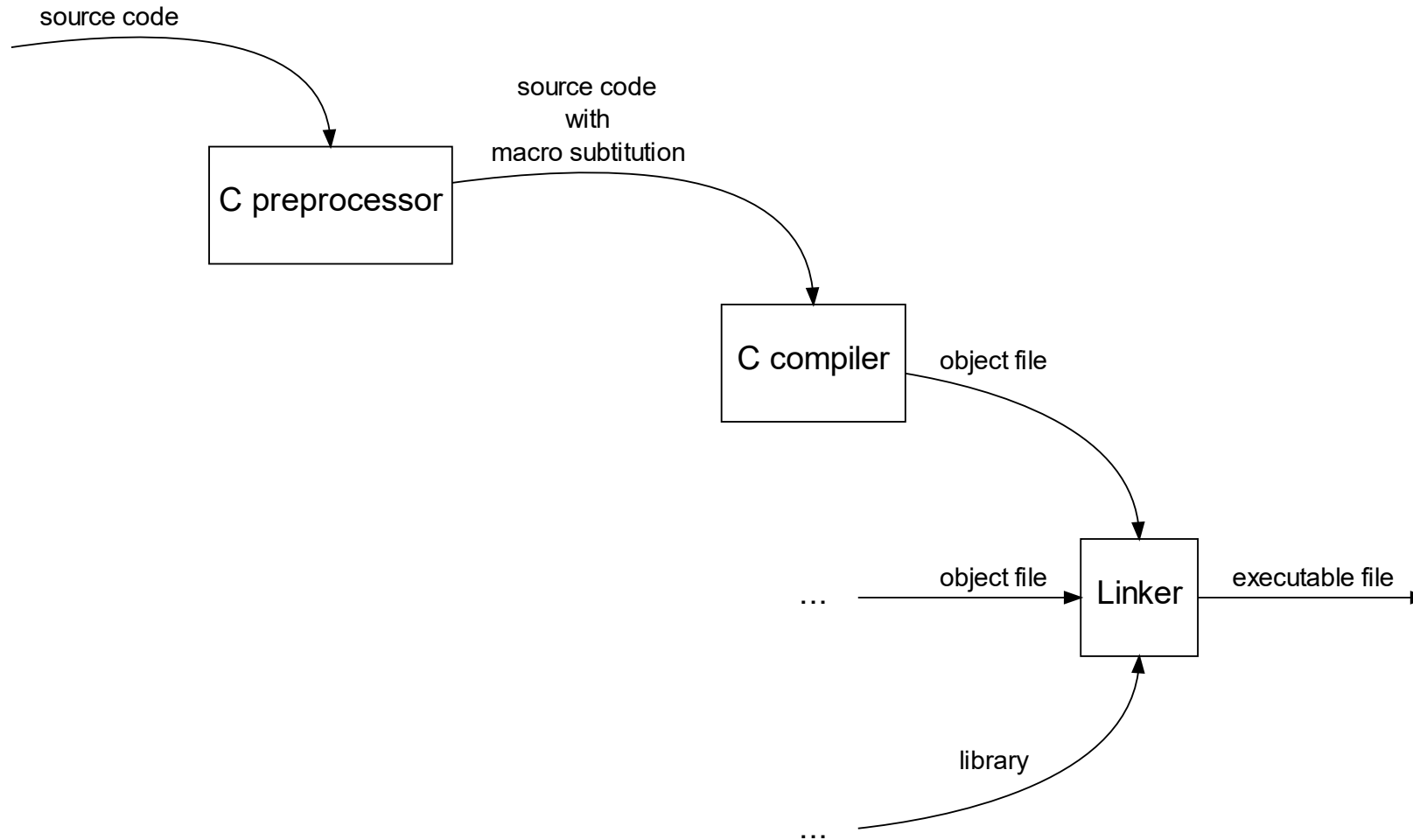
- Satu program utama (*main program*)
- Beberapa modul yang lain

Program yang dibagi-bagi menjadi beberapa file seharusnya dapat dikompilasi terpisah. Setiap modul membentuk sebuah *object code*.

Pembuatan sebuah *executable code* dilakukan dgn melakukan *linking* terhadap sejumlah *object code* yang sudah dikompilasi.

Penghematan waktu dan duplikasi usaha (*reusability*).

Pemrosesan kode sumber dalam Bahasa C



Modul program dalam C (1/3)

Program utuh terdiri dari 4 kelompok file

1. File *header* dengan nama `xxx.h` di folder `src/`

Untuk setiap type dan primitifnya, ada sebuah file *header*.

Contoh: untuk ADT Time dan ADT Date ada 2 buah file header, yaitu `time.h` dan `date.h`

Fungsi selektor (`get*`, `set*`) dapat digantikan dengan macro berparameter. Misalnya untuk selektor `HOURS(t)`, `MINUTES(t)`, dan `SECONDS(t)` dituliskan sebagai:

```
#define HOURS(t) (t).hours  
#define MINUTES(t) (t).minutes  
#define SECONDS(t) (t).seconds
```

Modul program dalam C (2/3)

2. File body dengan nama `xxx.c` di folder `src/`

Berisi realisasi dari prototype yang didefinisikan dalam file *header*.

Akan ada sebuah `xxx.c` untuk setiap `xxx.h`

Contoh : untuk file header `Time.h` dan `Date.h` akan ada file body `Time.c` dan `Date.c`

Modul program dalam C (3/3)

3. File main (driver) di folder `src/`

Berisi program utama dan prosedur/fungsi lain yang hanya dibutuhkan oleh main

Misalnya diberi nama `main.c`

4. File unit test di folder `tests/`

Berisi beberapa *test case* untuk setiap prosedur/fungsi yang ada di header

Misalnya, menggunakan library check, diberi nama `check_time.c`

Program Uteh akan terdiri dari sebuah `main.c`, sebuah `xxx.h`, `xxx.c`, dan `check_xxx.c`

File Header

```
/* File : xxx.h */
/* Deskripsi : keterangan isi file header */
/* Isi : deklarasi konstanta, type dan prototype */
/* File header TIDAK BOLEH mengandung deklarasi variabel! */

#ifdef xxx_h
#define xxx_h
/* Bagian I : berisi deklarasi konstanta */

/* Bagian II : berisi deklarasi type */

/* Bagian III : berisi deklarasi prototype prosedur & fungsi */
/* yg merupakan primitif type tsb */
/* Kelompokkan fungsi dan prosedur sesuai standar di kelas */
/* Mis.: konstruktor, selektor, predikat, operator relasional*/
/* operator aritmatika, operator lain, dsb */

#endif
```


File Body

```
/* File : xxx.c */  
/* Deskripsi : keterangan isi file body */  
/* Isi : realisasi/ kode program dari semua prototype */  
/* yg didefinisikan pada xxx.h */  
/* Untuk sebuah mesin akan mengandung deklarasi variabel */  
/* state dari mesin tsb */  
  
# include "xxx.h"  
  
/* Realisasi kode program, sesuai urutan pada xxx.h */  
  
/* Copy dari xxx.h, kemudian edit */
```

File Main Program

```
/* File : main_xxx.c */
/* Deskripsi: program utama & semua nama lokal thd persoalan */

#include "xxx.h"
/* include file lain yg diperlukan */
/* Bagian I : berisi kamus GLOBAL dan prototype */
/* deklarasi semua nama dan prosedur/fungsi global */

/* Bagian II : program utama */
int main() {
    /* Kamus lokal terhadap main */

    /* Algoritma */

    return 0;
}

/* Bagian III : berisi realisasi kode program yang merupakan */
/* BODY dari semua prototype yg didefinisikan pada file ini */
/* yaitu pada bagian I, dengan urutan-urutan yang sama */
/* Copy prototype, kemudian edit */
```

File Unit Test

```
/* File : check_xxx.c */
/* Deskripsi: unit test untuk modul xxx */

#include <check.h>
#include "../src/xxx.h"
/* include file lain yg diperlukan */

START_TEST(test_xxx_namafungsiyangdittest) {
    /* Bagian I: berisi test untuk sebuah fungsi dari modul xxx */
} END_TEST

Suite *xxx_suite(void) {
    /* Bagian II: mengumpulkan semua test menjadi satu test suite */
}

int main(void) {
    /* Bagian III: menjalankan unit test */
}
```

Penyimpanan Modul Program

Untuk setiap modul xxx.h dan xxx.c dibuat unit test untuk menguji setiap fungsi/prosedur yg dibuat.

Setiap paket yg terdiri dari xxx.h, xxx.c, unit test (misal: check_xxx.c), dan hasil test disimpan dalam satu direktori.

PR: baca dan praktikkan [Check 0.15.2: 3 Tutorial: Basic Unit Testing](#).

